

บทที่ 4

ระบบเครือข่ายกับไคลเอนต์เซิร์ฟเวอร์

4.1 ความรู้เบื้องต้นกับระบบเครือข่าย

ทุกวันนี้ เครือข่ายอินเทอร์เน็ต ถือเป็นเครือข่ายสาธารณะที่เปิดโอกาสให้ทุกคนสามารถที่จะเข้าสู่ระบบเครือข่ายได้อย่างกว้างขวาง โดยทางกายภาพแล้วเครือข่ายอินเทอร์เน็ตถือว่าเป็นส่วนหนึ่งของทรัพยากรที่ถูกใช้สำหรับการส่งผ่านข้อมูลในเครือข่ายคอมพิวเตอร์ทั้งหมดที่มีอยู่ในปัจจุบัน ซึ่งโดยส่วนมากเครื่องคอมพิวเตอร์ต่างๆ ที่จะเชื่อมต่อเข้าสู่เครือข่ายอินเทอร์เน็ตนั้น จะต้องใช้มาตรฐานของรูปแบบในการสื่อสาร หรือโปรโตคอล (Protocol) คือ TCP/IP (Transmission Control Protocol / Internet Protocol) นอกจากนี้ยังได้มีการพัฒนาและประยุกต์ระบบเครือข่ายอินเทอร์เน็ต ให้เป็นระบบเครือข่ายในลักษณะของเครือข่ายอินทราเน็ต (Intranet) และเครือข่ายเอ็กทราเน็ต (Extranet) ซึ่งจะมีโปรโตคอลหลักเป็น TCP/IP เช่นเดียวกัน

4.1.1 ระบบหมายเลขไอพี (IP Address)

เครื่องคอมพิวเตอร์ทุกเครื่องในเครือข่ายอินเทอร์เน็ตจะสามารถติดต่อถึงกันได้โดยเครื่องคอมพิวเตอร์ ต้นทางจะระบุที่อยู่ของเครื่องคอมพิวเตอร์ปลายทางที่ต้องการติดต่อด้วย ซึ่งที่อยู่ ของเครื่องคอมพิวเตอร์นี้จะถูกกำหนดใช้มาตรฐานที่เรียกว่า หมายเลขไอพี (IP Address) ซึ่งเป็นข้อมูลตัวเลขขนาด 32 บิต อันประกอบด้วยตัวเลขจำนวนเต็ม 4 กลุ่ม กลุ่มละ 8 บิต โดยใน แต่ละกลุ่มจะสามารถแทนค่าเป็นจำนวนเต็มที่อยู่ในช่วง 0-255 ได้ และใช้จุดเป็นตัวแบ่งกลุ่ม เช่น 161.246.6.2 203.166.54.23

หมายเลขไอพีหนึ่งๆ จะแสดงถึงข้อมูลด้านเครือข่ายอยู่ 2 ส่วนด้วยกัน ส่วนแรกเป็นส่วนแสดงหมายเลข เครือข่าย (Network Address) ส่วนที่สองเป็นส่วนแสดงหมายเลขคอมพิวเตอร์ (Host Address) ซึ่งเป็นที่อยู่ในเครือข่ายหมายเลขนั้น การจัดแบ่งหมายเลขไอพีสามารถแบ่งออกได้เป็น 5 คลาส (Class) คือ A, B, C, D และ E แต่โดยทั่วไปนั้นจะใช้งานหลักอยู่เพียง 3 คลาส ได้แก่ คลาส A, B และ C

เนื่องจากจำนวนคอมพิวเตอร์ทั่วโลกที่ทำการเชื่อมต่อเข้าสู่เครือข่ายอินเทอร์เน็ตได้เพิ่มขึ้นอย่างต่อเนื่อง ดังนั้นอีกในไม่ช้านี้ระบบหมายเลขไอพีที่ใช้ในปัจจุบันซึ่งเป็นข้อมูลตัวเลขขนาด 32 บิตจะไม่เพียงพอที่จะกำหนดให้แก่คอมพิวเตอร์ได้ทั่วโลก ดังนั้นจึงได้มีการเตรียมคิดค้นระบบหมายเลขไอพีแบบใหม่โดยขยายขนาดเป็นข้อมูลตัวเลขขนาด 128 บิต ซึ่งมีชื่อเรียกว่า Internet Protocol Version 6 หรือเรียกโดยย่อว่า IPv6

4.1.2 โปรโตคอลหลักที่ใช้ในการสื่อสารผ่านเครือข่ายอินเทอร์เน็ต

โปรโตคอล (Protocol) คือ ระเบียบวิธีการในการติดต่อสื่อสาร เมื่อมาใช้กับเทคโนโลยีสื่อสารโทรคมนาคม จึงหมายถึงขั้นตอนการติดต่อสื่อสาร ซึ่งรวมถึง กฎ ระเบียบ และข้อกำหนดต่าง ๆ รวมถึงมาตรฐานที่ใช้ เพื่อให้ตัวรับและตัวส่งสามารถดำเนินกิจกรรมทางด้านสื่อสารได้สำเร็จ

ในระบบอินเทอร์เน็ต จะใช้ภาษาสื่อสารมาตรฐาน คือ ทีซีพี/ไอพี (Transmission Control/Internet Protocol : TCP/IP) เป็นภาษาหลัก ดังนั้น หากเครื่องคอมพิวเตอร์ใดไม่ว่าจะเป็นเครื่อง Personal Computer, Macintosh, หรือเครื่องระดับมินิ จนไปถึงเมนเฟรม (Mainframe) มี TCP/IP นี้อยู่ ก็จะ สามารถเชื่อมโยงเข้าสู่อินเทอร์เน็ตได้

ในการส่งข้อมูลผ่านทาง TCP/IP นั้น TCP/IP จะทำการแบ่งข้อมูลนั้นออกเป็นส่วนย่อยๆ เรียกว่า แพ็กเก็ต (Packet) โดยแต่ละส่วนจะถูกเพิ่มข้อมูลบอกตำแหน่งต้นทาง และปลายทางที่จะส่งไว้ให้ TCP/IP เป็นโปรโตคอลที่รับประกันการรับ-ส่งข้อมูลระหว่างโฮสต์ มีกลไกในการตรวจสอบและยืนยันว่าข้อมูล จากต้นทางจะไปถึงปลายทางเสมอ โปรโตคอล TCP นั้น จะทำงานอยู่บนเลเยอร์ของ IP ซึ่งทำให้สามารถ รับส่งข้อมูลไปได้ทุกที่ในโลก นับเป็นจุดเด่นที่ทำให้โปรโตคอล TCP/IP เป็นที่นิยม

4.1.3 แพ็กเก็ต (Packets)

แพ็กเก็ต เป็นข้อมูลที่ส่งในระดับชั้นทรานสปอร์ตเลเยอร์ เป็นตัวกำหนดโครงสร้างของข้อมูลที่ ทำการรับส่งผ่านระบบเครือข่าย ซึ่งในแต่ละแพ็กเก็ตนั้น จะประกอบไปด้วยส่วนของ TCP/IP Header และส่วนข้อมูล ที่เราสามารถใส่เข้าไปในขณะเขียนโปรแกรมได้ โดยอาจเป็นลักษณะข้อมูลของสตริง หรือ เป็นโครงสร้างของข้อมูล

4.1.4 ซ็อกเก็ต (Sockets)

ซ็อกเก็ต จัดได้ว่าเป็นการสื่อสารระหว่างโปรเซส (Inter-Process Communication) ซ็อกเก็ตเป็น กลไกที่ทำงานในระดับล่าง จึงมีหน้าที่การใช้งานควบคุมมากมาย ซึ่งโดยปกติซ็อกเก็ตจะเป็นการสื่อสาร ผ่านระบบเครือข่าย ซึ่งอาจจะเป็น LAN หรืออินเทอร์เน็ต หรือแม้กระทั่งส่งกลับมาที่ไอพีของตนเองที่ พอร์ตต่างๆ ของโปรแกรมอื่นๆ ได้ จุดนี้เองที่เป็นลักษณะของการสื่อสารระหว่างโปรเซส หรือ Inter-Process Communication

ไลบรารีที่ใช้จัดการซ็อกเก็ต โดยส่วนมากที่มีการใช้งานกันมาก ก็คือ WinSock ซึ่งเป็นไลบรารีที่ ไมโครซอฟต์พัฒนาขึ้น โดยเริ่มต้นนั้นซ็อกเก็ตพัฒนามาจากเบอร์เกอร์ลีซ็อกเก็ต (Berkeley Socket) ซึ่ง ไลบรารีของ Winsock จะมีการให้ใช้งานเพื่อเขียนโปรแกรมติดต่อกับเน็ตเวิร์กได้ สามารถกำหนด โปรโตคอล ซึ่งโปรโตคอลนี้จะเป็นโปรโตคอลอะไรนั้น ก็ขึ้นอยู่กับว่าต้องการใช้งานในรูปแบบใด สำหรับการพัฒนาโปรแกรมประยุกต์ที่มีการติดต่อทางเครือข่ายนั้น โปรโตคอล TCP/IP ก็เป็นโปรโตคอล ที่ใช้งานกันอย่างแพร่หลาย เนื่องจากเป็นโปรโตคอลที่ยืนยันการรับส่งแพ็กเก็ต จึงเป็นที่น่าเชื่อถือ

4.1.5 บริการต่างๆ บนเครือข่ายอินเทอร์เน็ต

บริการต่างๆ บนเครือข่ายอินเทอร์เน็ต นั้นมีมากมาย ในที่นี้จะขอกล่าวเฉพาะบริการที่เกี่ยวข้อง กับโครงงานนี้ ได้แก่

- บริการรับส่งจดหมายอิเล็กทรอนิกส์ (Electronic Mail หรือ E-Mail) การใช้บริการรับส่ง จดหมายอิเล็กทรอนิกส์ จะมีลักษณะรูปแบบเช่นเดียวกับการส่งจดหมายทางไปรษณีย์ เพียงแต่จะ

เปลี่ยนเป็นการส่งจากระบบคอมพิวเตอร์ของผู้ส่ง ไปยังระบบคอมพิวเตอร์ของผู้รับ ผ่านทางเครือข่ายอินเทอร์เน็ต โดยเราสามารถส่งเอกสารจดหมายที่อยู่ในรูปแบบของดิจิทัล ไม่ว่าจะเป็น ตัวหนังสือ รูปภาพกราฟิกส์ วิดีโอ โปรแกรม หรือเพิ่มข้อมูลประเภทต่างๆ ได้ โดยจะดำเนินการจัดส่งไปยังผู้รับภายในเวลาเพียงไม่กี่นาทีเท่านั้น จะเห็นได้ว่าการรับส่งจดหมายอิเล็กทรอนิกส์ จะทำได้อย่างสะดวก รวดเร็วและประหยัดค่าใช้จ่ายกว่าการส่งจดหมายธรรมดาหรือ EMS เป็นอย่างมาก ทำให้บริการนี้กลายเป็นบริการที่มีประโยชน์อย่างยิ่งโดยเฉพาะในการติดต่อทางธุรกิจในยุคปัจจุบัน

- บริการติดต่อสนทนาออนไลน์ (Chat) นอกจากการติดต่อถึงกัน ในลักษณะจดหมายอิเล็กทรอนิกส์แล้ว บริการที่ได้รับความนิยมอีกอย่างคือการสนทนาในรูปแบบของการตอบสนองอย่างทันที (Interactive) ผู้ใช้สามารถที่จะคุยโต้ตอบกันได้เสมือนกับได้มาสนทนากันในระยะใกล้ๆ โดยในปัจจุบันบริการนี้สามารถสนทนาโดยใช้ทั้งข้อความตัวอักษรภาพและเสียงได้ในขณะเดียวกัน

4.2 ไคเร็กซ์เพลย์ (DirectPlay)

ไมโครซอฟต์ ไคเร็กซ์เพลย์ แอปพลิเคชัน โปรแกรมมิ่ง อินเตอร์เฟซ (Application Programming Interface : API) เป็นคอมโพเนนต์ตัวหนึ่งในไคเร็กซ์เอ็กซ์ที่ช่วยให้โปรแกรมเมอร์สามารถเขียนโปรแกรมติดต่อกับเน็ตเวิร์ก เช่น โปรแกรมประยุกต์แบบผู้เล่นหลายคน (Multiplayer Application)

ไคเร็กซ์เพลย์จะทำการสร้างเลเยอร์ขึ้นมาหนึ่งเลเยอร์ ซึ่งเป็นการแบ่งตัวอย่างเด็ดขาดจากส่วนพื้นฐานของเครือข่าย โดยมีจุดประสงค์หลักเพื่อให้แอปพลิเคชันสามารถเรียกใช้งานได้อย่างง่ายดาย โดยไม่สนใจชั้นของการติดต่อจริง และให้หน้าที่ในการจัดการในส่วนปลีกย่อยของการติดต่อระหว่างเครือข่ายเป็นหน้าที่ของไคเร็กซ์เพลย์

4.2.1 ข้อดีของไคเร็กซ์เพลย์

ข้อดีของไคเร็กซ์เพลย์ คือ ความง่ายภายในการจัดการในส่วนของการพัฒนาโปรแกรมประยุกต์แบบผู้เล่นหลายคน ดังนี้

- นักพัฒนาแอปพลิเคชันไม่ต้องสนใจว่าการติดต่อนั้นจะเป็นการติดต่อระหว่างโมเด็มกับโมเด็ม หรือว่าเป็นการติดต่อระหว่างอินเทอร์เน็ตเคเบิล หรืออาจจะเป็นอุปกรณ์ใหม่ๆ เนื่องจากไคเร็กซ์เพลย์จะสร้างชั้นของการติดต่อแบบ Abstract ขึ้นมา

- รองรับการสร้างและจัดการกับเซชชั่นทั้งแบบ peer-to-peer และแบบ client/server
- บริหารและจัดการข่าวสาร(Message) ระหว่างสมาชิกในเซชชั่น บนระบบเครือข่ายที่ต่างกัน และมีข้อจำกัดของเครือข่ายที่ต่างกัน

- ทำให้แอปพลิเคชันสามารถติดต่อกันกับล็อบบี้(Lobby) ได้
- ทำให้ผู้ใช้สามารถติดต่อสื่อสารระหว่างกัน โดยใช้เสียงคุยกันได้

4.2.2 การสร้างและจัดการกับเซสชัน

เซสชันของโปรแกรมเป็นอินสแตนซ์ (instance) ของโปรแกรมประยุกต์แบบผู้เล่นหลายคน ในหนึ่งเซสชัน จะมีผู้ใช้มากกว่าสองคนขึ้นไป ซึ่งในเวลาเดียวกันแต่ละคนจะใช้งานโปรแกรมประยุกต์ที่เครื่องของตน โดยผู้ใช้จัดเป็นส่วนประกอบหนึ่ง และถูกกำหนดขึ้นโดยโปรแกรม ซึ่งผู้ใช้อาจจะเป็นได้มากกว่าผู้ใช้หนึ่งคนในโปรแกรมประยุกต์ได้ สำหรับตัวโปรแกรมประยุกต์นี้ ก็จะใช้ฟังก์ชันหน้าที่ในการจัดการเกี่ยวกับผู้ใช้แต่ละคน โดยใช้อินเทอร์เฟซของไคลเอนต์เซิร์ฟเวอร์

ขั้นตอนแรกในการสร้างเซสชัน คือ การเก็บกลุ่มของผู้ใช้ โดยมีทฤษฎีเบื้องต้น 2 ทฤษฎีด้วยกัน คือ

- โปรแกรมประยุกต์บางโปรแกรม จะถูกจัดเตรียมโดยล็อบบี้แอปพลิเคชัน (Lobby Application) ซึ่งทำงานอยู่บนคอมพิวเตอร์ปลายทาง ซึ่งวิธีนี้จะนิยมใช้มากในโปรแกรมประยุกต์แบบพื้นฐานบนอินเทอร์เน็ต (Internet-based)

- โปรแกรมประยุกต์บางโปรแกรม ถูกจัดการเตรียมเซสชันโดยมีผู้ใช้แต่ละเครื่องในเครือข่ายติดต่อกัน โดยวิธีนี้มีข้อจำกัด ว่ากลุ่มของผู้ใช้ที่ใช้งานโปรแกรม จะต้องอยู่บนเครือข่ายเดียวกัน

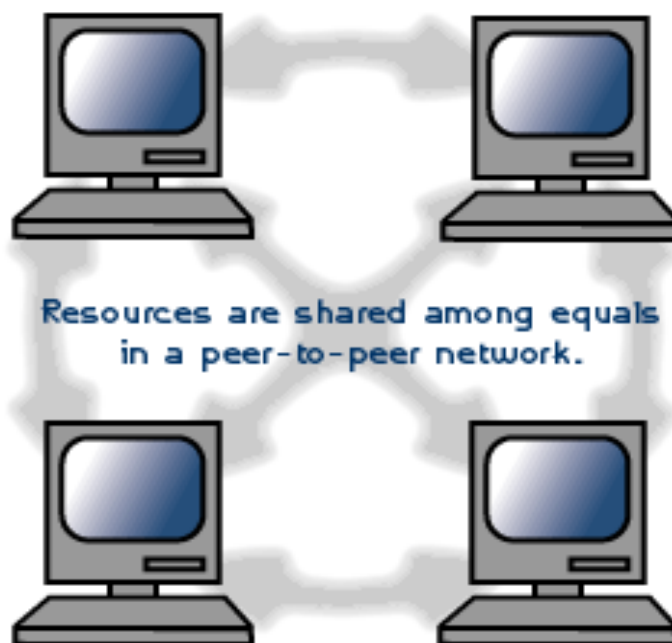
เมื่อเซสชันของโปรแกรมได้ถูกสร้างขึ้นมาแล้ว การทำงานของโปรแกรมก็จะเริ่มขึ้น ซึ่งในขณะที่เซสชันทำงานอยู่นั้น ผู้ใช้อาจจะออกจากเซสชัน หรือมีผู้ใช้คนใหม่เข้ามาก็ได้ ขึ้นอยู่กับลักษณะของโปรแกรมที่เขียนขึ้นมานั้นๆ

ในโปรแกรมประยุกต์แบบผู้เล่นหลายคน ผู้ใช้แต่ละคน จะสามารถใช้งานไปพร้อมๆ กับผู้ใช้คนอื่นๆ ในเซสชันเดียวกันได้ การจัดการการสื่อสารระหว่างผู้ใช้ในเซสชันจึงจำเป็นต้องมีการสื่อสารที่ต่อเนื่องของข้อมูลของผู้ใช้ที่รับส่งไปมา เช่น ในเวลาที่ผู้ใช้มีการเดิน จะมีข้อมูลส่งไปบอกตำแหน่งใหม่ของผู้ใช้คนที่เดินอยู่ ให้กับผู้ใช้งานคนอื่นๆ ทุกคนในเซสชันนั้นรับทราบ จึงกล่าวได้ว่า หน้าที่หลักของไคลเอนต์เซิร์ฟเวอร์ คือ เป็นส่วนหนึ่งของเอพีไอที่สนับสนุนการส่งข้อมูลระหว่างคอมพิวเตอร์ในเซสชันให้เป็นไปอย่างมีประสิทธิภาพ และมีความยืดหยุ่น

รูปแบบวิธีการส่งข่าวสารระหว่างเซสชัน แบ่งไว้เป็น 2 แบบ ดังนี้

4.2.2.1 การติดต่อสื่อสารข้อมูลแบบเครื่องต่อเครื่อง (Peer-to-Peer Topology)

โปรแกรมประยุกต์ที่มีการติดต่อเครือข่ายแบบเครื่องต่อเครื่อง (Peer-to-Peer Application) จะประกอบด้วยเครื่องคอมพิวเตอร์ของผู้ใช้แต่ละคน เชื่อมต่อกันในเครือข่าย ดังรูปที่ 4-1 นี้



รูปที่ 4-1 การเชื่อมต่อแบบเครื่องต่อเครื่อง โดยในรูปนี้จะมีเครื่องคอมพิวเตอร์ที่ใช้งานอยู่ 4 เครื่อง

เมื่อมีการเปลี่ยนแปลงต่างๆ จะมีการสื่อสารระหว่างเครื่องเกิดขึ้นโดยตรง เพื่อบอกถึงความเปลี่ยนแปลงที่เกิดขึ้นนั้น เช่น ถ้ามีผู้ใช้งานหนึ่งพิมพ์ข้อความสนทนา ก็จะมีการส่งข้อมูลไปบอกเครื่องอื่นๆ อีกสามเครื่องที่ติดต่อกันแบบเครื่องต่อเครื่องนี้อยู่ เพื่อบอกให้ทราบว่าข้อความสนทนาเข้ามา แล้วแสดงให้ผู้ใช้งานได้เห็น

โดยปกติ จะมีการจัดการและเรียกผ่านล็อบบี้ไคลเอนต์บนเครื่องฝั่งผู้ใช้ มีหลักการเบื้องต้นในการที่ล็อบบี้ไคลเอนต์จัดการกับเซชัน 2 วิธี ดังนี้

- ล็อบบี้ไคลเอนต์ ติดต่อสื่อสารโดยตรงกับไคลเอนต์ของผู้ใช้คนอื่นๆ วิธีนี้สามารถนำไปใช้กับการจัดการการติดต่อระหว่างผู้ใช้งานเครือข่ายที่อยู่ในชั้นเนทวงแลนเดียวกันได้
- ล็อบบี้ไคลเอนต์ ทำหน้าที่คล้ายเป็นตัวเชื่อมต่อกับล็อบบี้เซิร์ฟเวอร์ ซึ่งทำงานอยู่บนเครื่องคอมพิวเตอร์ปลายทาง ซึ่งวิธีนี้เป็นลักษณะของโปรแกรมที่ติดต่อกับเครือข่ายบนอินเทอร์เน็ต

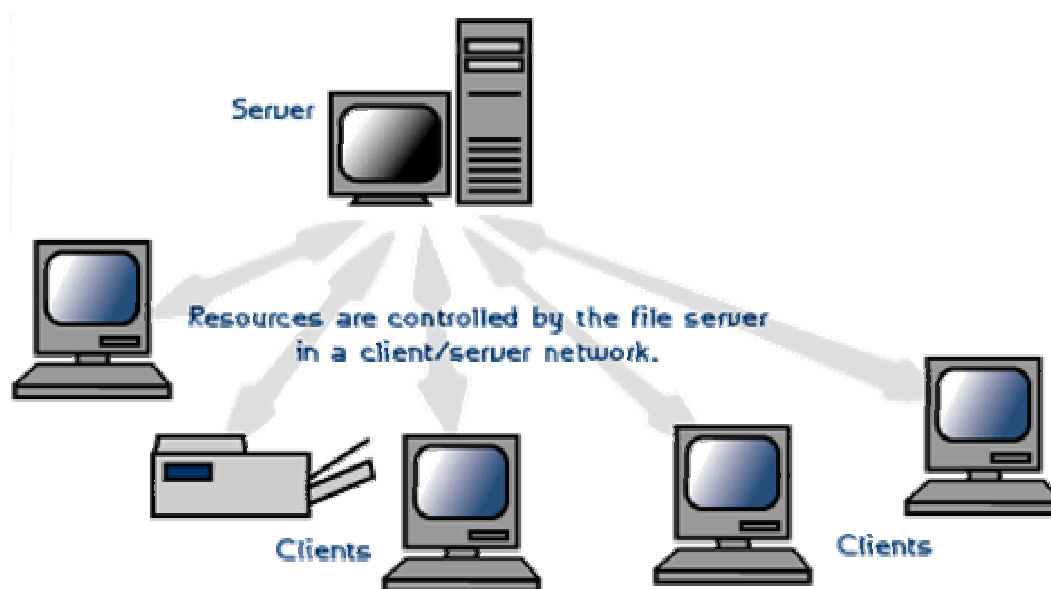
เมื่อเซชันถูกสร้างและทำงานขึ้น โดยส่วนมากข้อมูลที่ทำการรับส่งจะเป็นข้อมูลที่รับส่งระหว่างผู้ใช้งานกับผู้ใช้งานตัวเอง ถ้ามีลักษณะเป็นล็อบบี้เซิร์ฟเวอร์ ส่วนของล็อบบี้เซิร์ฟเวอร์จะมีหน้าที่แค่จัดการเมื่อมีผู้ใช้ออกจากเกม หรือเมื่อมีผู้ใช้งานใหม่เข้ามาเท่านั้น นอกจากนี้ เซิร์ฟเวอร์จะไม่สนใจข้อมูลที่ส่งผ่านไปมา

ในรูปแบบแบบเครื่องต่อเครื่อง (Peer-to-Peer) นี้ เซิร์ฟเวอร์นั้นเหมือนกับว่าไม่ได้มีความเกี่ยวข้องกับเกม ผู้ใช้คนหนึ่งอาจจะมีหน้าที่เป็นโฮสต์หรือเป็นเซิร์ฟเวอร์ ซึ่งมีหน้าที่เพียงแค่นำผู้ใช้งานใหม่เข้ามาสู่เกมเท่านั้น เมื่อผู้ใช้ที่ทำหน้าที่เสมือนเป็นโฮสต์นี้เลิกการติดต่อไป ผู้ใช้คนใหม่ที่ยังอยู่ในเซชันนี้จะมาทำหน้าที่เป็นโฮสต์ใหม่แทน

ข้อดี	ข้อเสีย
1.แม้ว่าผู้ใช้บางคนจะเลิกการติดต่อไปแต่โปรแกรมประยุกต์แบบเครื่องต่อเครื่องนี้ ก็ยังสามารถทำงานต่อไปได้	1.จำนวนผู้เล่นจะถูกจำกัดลงโดยความสามารถของระบบเครือข่าย
2.ไม่จำเป็นต้องใช้เซิร์ฟเวอร์ที่มีความเร็วสูงมาเป็นส่วนหนึ่งของระบบ	2.เกิดความคับคั่งในระบบเครือข่าย
3.ง่ายต่อการพัฒนา	3.ข้อมูลมีความเสี่ยงสูงในการที่จะไม่ได้รับ

ตารางที่ 4-1 ข้อดีข้อเสียของโปรแกรมประยุกต์แบบเครื่องต่อเครื่อง (Peer-to-Peer)

4.2.2.2 การติดต่อสื่อสารข้อมูลแบบไคลเอนต์/เซิร์ฟเวอร์



รูปที่ 4-2 การเชื่อมต่อแบบไคลเอนต์/เซิร์ฟเวอร์ โดยมีผู้ใช้จำนวน 4 เครื่อง

การเปลี่ยนแปลงข้อมูลต่างๆ ของโปรแกรม รวมไปถึงสแตกการทำงานต่างๆ จะกระทำโดยผู้ใช้แต่ละคนจะติดต่อกันผ่านเครื่องเซิร์ฟเวอร์ โดยเซิร์ฟเวอร์มีหน้าที่ส่งข้อมูลต่างๆ ไปยังทุกคน เช่น เมื่อมีผู้ใช้งานหนึ่งคน จะมีการส่งข้อมูลไปยังเซิร์ฟเวอร์ จากนั้นเซิร์ฟเวอร์จะส่งข้อมูลไปให้ผู้ใช้งานอื่นๆ ว่าผู้ใช้งานแรกมีการเปลี่ยนตำแหน่งไปแล้ว โดยเซิร์ฟเวอร์ จะทำหน้าที่ต่างๆ ดังนี้

- ทำหน้าที่เสมือนเป็นตัวส่งผ่านข้อมูล เมื่อเครื่องคอมพิวเตอร์ลูกข่ายใดต้องการส่งข้อมูล ก็จะส่งข้อมูลไปที่เครื่องเซิร์ฟเวอร์ จากนั้น เซิร์ฟเวอร์จะกระจายข้อมูลไปยังเครื่องคอมพิวเตอร์ลูกข่ายอื่นๆ วิธีนี้จะช่วยลดความคับคั่งของข้อมูลได้เหมาะสมสำหรับโปรแกรมใหญ่ๆ
- ถ้าเปรียบกับรูปแบบการติดต่อสื่อสารข้อมูลแบบเครื่องต่อเครื่อง เครื่องเซิร์ฟเวอร์ในการสื่อสารแบบนี้ จะทำหน้าที่เสมือนเป็นโฮสต์ในรูปแบบการติดต่อสื่อสารข้อมูลแบบเครื่องต่อเครื่อง

● ทำหน้าที่จัดการคำนวณผลเพื่อนำไปแสดงผลของโปรแกรมที่ฝั่งลูกค้า โปรแกรมที่มีขนาดใหญ่ส่วนมากการจัดการต่างๆ จะถูกนำไปไว้ยังเครื่องเซิร์ฟเวอร์ ส่วนเครื่องไคลเอ็นท์จะทำหน้าที่แสดงผลเท่านั้น

โปรแกรมแบบไคลเอ็นท์/เซิร์ฟเวอร์นั้นโดยปกติแล้วจะถูกจัดการ และเรียกใช้งานผ่านล็อบบี้ไคลเอ็นท์ ที่อยู่ในเครื่องของผู้ใช้งาน ตัวล็อบบี้ไคลเอ็นท์นั้นจะทำหน้าที่เชื่อมต่อไปยังล็อบบี้เซิร์ฟเวอร์ ซึ่งโดยปกติแล้วจะทำงานอยู่บนเครื่องเดียวกับเครื่องคอมพิวเตอร์ที่ทำหน้าที่เป็นโฮสต์ของโปรแกรม เมื่อโปรแกรมถูกเรียกขึ้นมาทำงาน เครื่องเซิร์ฟเวอร์จะกลายเป็นโฮสต์ของโปรแกรม และทำงานต่างๆ

ข้อดี	ข้อเสีย
1. รูปแบบไคลเอ็นท์/เซิร์ฟเวอร์ จะมีประสิทธิภาพการทำงานที่ดีกว่าแบบเครื่องต่อเครื่อง ซึ่งจะมีการเพิ่มขึ้นของข้อมูลเมื่อมีผู้ใช้เพิ่มขึ้นมา ในอัตราเชิงเส้น (Linear Increase) ซึ่งถ้าเป็นไคลเอ็นท์/เซิร์ฟเวอร์นั้น การส่งข้อมูล จะส่งจากเซิร์ฟเวอร์กระจายไปให้เครื่องอื่นๆ ไม่ใช่แบบส่งเครื่องต่อเครื่อง กับทุกเครื่อง	1. เครื่องที่จะรองรับ การทำงาน ของโปรแกรมเซิร์ฟเวอร์นั้น ต้องเป็นเครื่องที่มีฮาร์ดแวร์ประสิทธิภาพ และความเร็วสูง เพื่อรองรับการติดต่อเข้ามาของผู้ใช้จำนวนมากได้ ซึ่งเครื่องเหล่านี้จะมีราคาสูงมาก
2. การใช้งานโปรแกรมลูกค้า จะไม่ถูกจำกัดโดยประสิทธิภาพของเครื่องผู้ใช้ เนื่องจากอาจจะเขียนโปรแกรมให้มีการประมวลผลหลักๆ ที่ฝั่งเซิร์ฟเวอร์ และให้โปรแกรมลูกค้ามีหน้าที่เพียงแค่แสดงผล	2. ต้องใช้ความเร็วในระบบเครือข่าย มากกว่าแบบเครื่องต่อเครื่อง (Peer-to-Peer)
3. การแก้ไขหน้าที่การทำงานของโปรแกรม จะง่ายขึ้น ถ้าสามารถแก้ไขที่การทำงานของเซิร์ฟเวอร์เพียงทีเดียว ไม่ต้องไปแก้ที่โปรแกรมไคลเอ็นท์ ซึ่งจะลำบากผู้ใช้ที่ต้องมาโหลดโปรแกรมไปใหม่	3. หากเซิร์ฟเวอร์ หยุดทำงาน หรือหลุดออกจากระบบไป โปรแกรมลูกค้า จะไม่สามารถทำงานต่อได้เลย

ตารางที่ 4-2 ข้อดีข้อเสียของโปรแกรมประยุกต์แบบไคลเอ็นท์/เซิร์ฟเวอร์

4.2.3 ไคเร็กซ์เพลย์กับการสื่อสารในระบบเครือข่าย (DirectPlay Network Communication)

หัวใจหลักของไคเร็กซ์เพลย์ คือ การรองรับการจัดการส่งข้อมูล ได้อย่างยืดหยุ่นและมีประสิทธิภาพ และสามารถทำได้ง่ายโดยอาศัย โดยการเรียกผ่านเอพีไอของไคเร็กซ์เพลย์ โดยไม่สนใจว่าใช้ระบบเครือข่ายชนิดใด ไคเร็กซ์เพลย์นั้นรองรับการติดต่อทั้งบน TCP/IP, IPX, Modem และ การติดต่อโดยตรง (Serial Links)

4.2.3.1 โพรโตคอลที่ใช้ในการสื่อสารของไคเร็กซ์เพลย์ (DirectPlay Transportation Protocol)

หัวใจของความสามารถของไมโครซอฟต์ไคเร็กซ์เพลย์ คือ ไคเร็กซ์เพลย์โพรโตคอล ซึ่งชั้นทรานสปอร์ตเลเยอร์โพรโตคอล (Transport-Layer Protocol) นี้ ได้รับการปรับปรุงใหม่ทั้งหมดในไคเร็กซ์เอ็กซ์เวอร์ชัน 8 นี้ และสามารถใช้ได้กับการสื่อสารทุกรูปแบบ ไคเร็กซ์เพลย์โพรโตคอลนี้มีจุดประสงค์เพื่อให้การส่งผ่านข้อมูลระหว่างต้นทางกับปลายทางสามารถทำได้ง่ายโดยโดยไม่ต้องสนใจว่ามีการกระทำอย่างไรในขั้นตอนเหล่านั้น โพรโตคอลดังกล่าวนี้จะมีหลายๆ ทางเลือกซึ่งจำเป็นต่อโปรแกรมหลายประเภทที่มีหลายผู้ใช้ให้เลือกได้แก่

- การส่งข้อมูลแบบเชื่อถือได้ (Reliable) และเชื่อถือไม่ได้ (Unreliable) การส่งแบบเชื่อถือได้ ซึ่งจะมีการรับรองจนกว่าปลายทางจะได้รับข้อมูล
- การสื่อสารแบบต่อเนื่อง (Sequential) และแบบไม่ต่อเนื่อง (non-Sequential) การส่งแบบต่อเนื่อง จะส่งไปยังปลายทางตามลำดับที่ได้รับมา
- การแบ่งข้อมูลออก (Fragmentation) และการประกอบเป็นแบบเดิม (Reassemble) ถ้าขนาดของข้อมูลเกินกว่าความสามารถของเครือข่ายที่จะส่งไปได้ ไคเร็กซ์เพลย์จะทำการแยกออกเป็นหลายๆ ให้แล้วจึงนำมาประกอบกันใหม่เมื่อถึงปลายทาง ซึ่งกระบวนการนี้ทำงานในระดับชั้นเน็ตเวิร์กเลเยอร์
- การควบคุมความคับคั่งของข้อมูล (Congestion Control) ไคเร็กซ์เพลย์จะทำการควบคุมข้อมูลที่ส่งออกไป ให้อยู่ในระดับที่เครื่องปลายทางสามารถจะรับได้ ลักษณะนี้จะช่วยป้องกันไม่ให้ข้อมูลล้นเกินกว่าจะสามารถประมวลผลทันได้ที่เครื่องปลายทาง
- ระดับความสำคัญของข้อมูล (Send Prioritization) เพื่อเป็นการรับประกันว่าข้อมูลที่สำคัญมากจะได้รับการส่งไปก่อน ไคเร็กซ์เพลย์สามารถจัดระดับความสำคัญของข้อมูลที่ส่งไปเป็นแบบ ต่ำ ปานกลาง และสูงได้ ข้อมูลที่สำคัญจะได้รับการส่งไปก่อน
- ระยะเวลาในการส่งข้อมูล (Message Timeouts) เพื่อป้องกันไม่ให้ต้องรอข้อมูลตอบรับจากเครื่องปลายทาง หากว่ามีการผิดพลาดเนื่องจากข้อมูลไปไม่ถึงปลายทาง ไคเร็กซ์เพลย์จะอนุญาตให้มีการกำหนดระยะเวลาในการรอไว้ได้ หากเกินเวลาแล้วข้อมูลที่กำลังจะส่งจะถูกนำออกจากคิว โดยไม่สนใจว่าส่งไปแล้วหรือยัง

4.2.3.2 การอ้างที่อยู่ของไคเร็กซ์เพลย์ (DirectPlay Addresses)

ในการส่งข้อมูล ผู้ที่อยู่ในระบบต้องมีแอดเดรสที่ไม่ซ้ำกัน ซึ่งแอดเดรสนี้อาจจะอ้างตามเครื่องคอมพิวเตอร์ที่กำลังทำการรันโปรแกรมนั้น (Device Address) หรือเครื่องคอมพิวเตอร์ที่ต้องการติดต่อสื่อ

สารด้วย (Host Address) แอดเดรสในไคลเอนต์เพลย์นั้นจะอยู่ในรูปแบบของข้อความ URL โดยข้อความดังกล่าวจะประกอบไปด้วย Header, ตัวแบ่ง และข้อความที่ประกอบด้วยข้อมูลต่างๆ ตามรูปแบบนี้ x-DirectPlay:/ [data string]

4.2.4 การติดต่อสื่อสารกันด้วยไคลเอนต์เพลย์ออบเจกต์ (Communication with DirectPlay Objects)

ไมโครซอฟต์ไคลเอนต์เพลย์นั้นประกอบไปด้วย COM ออบเจกต์หลายๆ ตัว โดยแต่ละตัวนั้นมีอินเตอร์เฟซ เพื่อให้สามารถจัดการกับส่วนต่างๆ ของไคลเอนต์เพลย์ได้

การติดต่อกับออบเจกต์ของไคลเอนต์เพลย์นั้น สามารถทำได้โดยการเรียกฟังก์ชันต่างๆ ผ่านทางอินเตอร์เฟซที่มีการจัดเตรียมไว้ให้

ไคลเอนต์เพลย์ติดต่อสื่อสารกับโปรแกรมประยุกต์ ผ่านทางสิ่งที่เรียกว่า Callback Function ที่อาจจะมีมากกว่าหนึ่งได้ ฟังก์ชันนี้จะคล้ายกับ Window Procedure โปรแกรมประยุกต์จะมีการจัดการกับ Callback Function และทำการส่งค่าพอยน์เตอร์ (Pointer) ที่ชี้ไปยังฟังก์ชันนั้นไปให้กับไคลเอนต์เพลย์ในระหว่างช่วงของการเริ่มต้น (Initialization) และเมื่อไคลเอนต์เพลย์ต้องการทำการติดต่อกับโปรแกรมประยุกต์นั้น ก็จะทำการเรียก Callback Function นี้และทำการส่งค่าข้อมูลสองชนิดดังนี้

- รหัสของข้อมูล (Message ID) ซึ่งทำหน้าที่กำหนดชนิดของข้อมูล
- พอยน์เตอร์ ที่ชี้ไปที่บล็อกของข้อมูล ปกติจะเป็นโครงสร้างที่มีการเก็บข้อมูลที่ต้องการไว้